



Deploy a RAG API to Kubernetes



Abhinave P.B

```
abhinave32@DESKTOP-32UV7P9: ~$ curl -X POST "http://127.0.0.1:34295/query?q=what+is+kubernetes"
{"answer": "Sure! Kubernetes is a container orchestration platform used for managing and deploying containerized applications on a large scale. It automates the process of starting, scaling, monitoring, and managing containerized services to create highly available, scalable, and robust application platforms. With Kubernetes, developers can build and deploy containers with zero downtime and zero manual intervention, ensuring maximum uptime, improved efficiency, and"}
abhinave32@DESKTOP-32UV7P9: ~$
```



Introducing Today's Project!

In this project, I will deploy a containerized application using Kubernetes. I am doing this project to gain hands-on experience with container orchestration and understand how applications are managed in a real-world cloud-native environment. Kubernetes will help me automate deployment, scaling, and networking of the application, giving me practical exposure to how modern distributed systems are built and managed.

Key services and concepts

Key concepts I learnt include container orchestration, self-healing, and service abstraction in Kubernetes. Kubernetes provides a platform to deploy, manage, and scale containerized applications automatically while ensuring reliability and availability. Deployments manage the desired state of the application by defining how many pod replicas should be running and by automatically recreating pods when failures occur. This ensures that the application continues to run without manual intervention. Services route network traffic to the appropriate pods using label selectors, providing a stable endpoint even when pods are recreated or their IP addresses change. This decouples the application from individual pod lifecycles and enables seamless communication within and outside the cluster.

Challenges and wins

This project took me approximately [X hours/days] to complete, including learning the basics of Kubernetes, deploying the application, and testing its behavior under failure conditions.



Abhinave P.B

NextWork Student

nextwork.org

The most challenging part was understanding how Pods, Deployments, and Services work together, especially debugging networking and service exposure in a Kubernetes environment. It was most rewarding to see the application remain available even after deleting a running pod, as this clearly demonstrated Kubernetes' self-healing capabilities in a real, practical scenario. Completing this project helped me gain hands-on experience with container orchestration and improved my confidence in deploying production-ready applications.

Why I did this project

I did this project because I wanted to learn how real-world applications are deployed and managed beyond just running containers locally. I wanted to understand how Kubernetes handles failures, networking, and application availability in a production-like environment. One thing I'll apply from this project is designing applications with resilience in mind, using Deployments and Services to ensure that applications can automatically recover from failures and remain accessible without manual intervention.



Setting Up My Docker Image

In this step, I'm setting up Docker image. I need a Docker image because Kubernetes Kubernetes can only run things that are already packaged.

```
abhinave32@DESKTOP-32UV7P9: $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
d36472772b0b	kicbase/stable:v0.0.49	"/usr/local/bin/entr..."	51 seconds ago	Up 50 seconds	127.0.0.1:65183->22/tcp, 127.0.0.1:65181->2376/tcp, 127.0.0.1:65188->5000/tcp, 127.0.0.1:65182->8443/tcp, 127.0.0.1:65179->32443/tcp
c3cc80c25950	Flask-app-pipeline-Flask-app	"python app.py"	3 weeks ago	Up 7 hours	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
ef2d86a8bc2d	mysql:8.0	"docker-entrypoint.s..."	3 weeks ago	Up 7 hours (healthy)	3306/tcp, 33060/tcp

```
abhinave32@DESKTOP-32UV7P9: $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
abhinave32/rag-app:latest	2e281b6fd4ec	1.34GB	445MB	U
docker/desktop-kubernetes:kubernetes.v1.34.1-cni.v1.7.1-critools.v1.33.0-cni-dockerd.v0.3.28-1-debian	12d6673564e0	592MB	185MB	U
docker/desktop-storage-provisioner:v2.0	115d77efe6e2	59.2MB	17.3MB	U
docker/desktop-storage-provisioner:v2.0	57d2b6ad1c0f	80.3MB	23.9MB	U
docker/desktop-vpnkit-controller-de331c27850b6cd897c84a9feca44a1af6e	7ecf557ea070	47MB	10.0MB	U
docker/desktop-vpnkit-controller:v4.0	bdaff3408b1c	52.6MB	11.9MB	U
Flask-app-pipeline-Flask-app:latest	43b8936737f3	588MB	158MB	U
per-10-k8s-minikube/kicbase@sha256:e6dadb1dc09cccd195c5805f65e2838406c36ef30c5a492f7fe805d0d36f4945	e6dadb1dc0	1.91GB	539MB	U
hello-world:latest	05813aedc15f	25.9KB	9.52KB	U
kicbase/stable:v0.0.49	c7ddf4818024	1.91GB	539MB	U
layer5/kanvas-docker-extension:stable-latest	2766ee4f3dee	47MB	11.5MB	U
layer5/meshery:kanvas-latest	42d73c7ec6d9	1.90GB	341MB	U
mysql:8.0	271aa4b4c844	1.08GB	247MB	U
ngrok/ngrok-docker-extension:1.0.0	a44979184584	29.3MB	8.43MB	U
registry.k8s.io/coredns/coredns:v1.12.1	e6c262566636	101MB	22.4MB	U
registry.k8s.io/etcd/v3.0.44-0	e36c80168342	273MB	74.3MB	U
registry.k8s.io/kube-apiserver:v1.34.1	b9d7c117f8ac	118MB	27.1MB	U
registry.k8s.io/kube-controller-manager:v1.34.1	2b4f47c1b01f5	181MB	22.8MB	U
registry.k8s.io/kube-proxy:v1.30.1	913cc83ce0b5	102MB	26MB	U
registry.k8s.io/kube-scheduler:v1.34.1	6e9fbc4e25a5	73.5MB	17.4MB	U
registry.k8s.io/pause:3.0	ee6521f290b2	1.06MB	318KB	U

```
abhinave32@DESKTOP-32UV7P9: $
```

What the Docker image contains

I ran docker images and saw my images. The image size was 1.34 GB The IMAGE ID was 2e281b6fd4ec

Installing Kubernetes Tools

In this step, I'm installing Minikube and kubectl. I need these tools because Minikube lets me run a local Kubernetes cluster on my computer, and kubectl allows me to interact with and manage that cluster from the command line. Together, they provide everything I need to deploy, test, and control my containerized application locally.

```
abhinave32@DESKTOP-32U0V7 Windows PowerShell
- cluster:
  certificate-authority: C:\Users\abhin\minikube\ca.crt
  extensions:
    - extension:
        last-update: Tue, 03 Feb 2026 15:30:42 IST
        provider: minikube.sigs.k8s.io
        version: v1.38.0
      name: cluster_info
      server: https://127.0.0.1:52045
    name: minikube
  contexts:
    - context:
        cluster: docker-desktop
        user: docker-desktop
      name: docker-desktop
    - context:
        cluster: minikube
        extensions:
          - extension:
              last-update: Tue, 03 Feb 2026 15:30:42 IST
              provider: minikube.sigs.k8s.io
              version: v1.38.0
            name: context_info
            namespace: default
            user: minikube
          name: minikube
        current-context: minikube
      kind: Config
      users:
        - name: docker-desktop
          user:
            client-certificate-data: DATA+OMITTED
            client-key-data: DATA+OMITTED
        - name: minikube
          user:
            client-certificate: C:\Users\abhin\minikube\profiles\minikube\client.crt
            client-key: C:\Users\abhin\minikube\profiles\minikube\client.key
PS C:\Users\abhin> minikube --version
Error: unknown flag: --version
See 'minikube --help' for usage.
PS C:\Users\abhin> kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
PS C:\Users\abhin> |
```

Verifying the tools are installed

I installed Minikube using the `winget install -e --id Kubernetes.minikube` command in Windows PowerShell. I installed kubectl using the `winget install -e --id Kubernetes.kubectl` command.

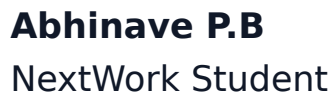


Abhinave P.B

NextWork Student

nextwork.org

I could tell both installations were successful because PowerShell displayed a “Successfully installed” message for each package, and the tools became available after restarting the shell, which allowed their versions to be verified using `minikube version` and `kubectl version --client`.



```

/ 271.45 MiB 2.60% 949.39 K > grc.io/k8s-minikube/kicbase... 33.77 MiB / 514.16 MiB 0.0% 2.49 Ki > preloaded-images-k8s-v18-v1... 7.64 MiB / 27
1.45 MiB 2.81% 949.39 K > grc.io/k8s-minikube/kicbase... 129.75 MiB / 514.16 MiB 0.8% 16.11 > preloaded-images-k8s-v18-v1... 8.14 MiB / 271.45
MiB 3.00% 969.78 M > grc.io/k8s-minikube/kicbase... 699.70 MiB / 514.16 MiB 0.12% 16.11 > preloaded-images-k8s-v18-v1... 8.54 MiB / 271.45
MiB 3.15% 969.78 M > grc.io/k8s-minikube/kicbase... 1.20 MiB / 514.16 MiB 0.23% 16.11 Ki > preloaded-images-k8s-v18-v1... 9.11 MiB / 271.45 MiB
3.96% 969.78 M > grc.io/k8s-minikube/kicbase... 2.25 MiB / 514.16 MiB 0.44% 249.28 K > preloaded-images-k8s-v18-v1... 9.48 MiB / 271.45 MiB 3.49%
1.83 MiB > grc.io/k8s-minikube/kicbase... 2.47 MiB / 514.16 MiB 0.48% 249.28 K > preloaded-images-k8s-v18-v1... 9.75 MiB / 271.45 MiB 3.60% 1.83
MiB > grc.io/k8s-minikube/kicbase... 3.14 MiB / 514.16 MiB 0.61% 249.28 K > preloaded-images-k8s-v18-v1... 9.95 MiB / 271.45 MiB 3.76% 1.83
MiB > grc.io/k8s-minikube/kicbase... 3.75 MiB / 514.16 MiB 0.73% 398.32 K > preloaded-images-k8s-v18-v1... 10.33 MiB / 271.45 MiB 3.80% 1.05 Mi
3.96% 1.83 MiB > grc.io/k8s-minikube/kicbase... 4.27 MiB / 514.16 MiB 0.83% 398.32 K > preloaded-images-k8s-v18-v1... 10.64 MiB / 271.45 MiB 3.92% 1.05
MiB > grc.io/k8s-minikube/kicbase... 4.41 MiB / 514.16 MiB 0.86% 398.32 K > preloaded-images-k8s-v18-v1... 10.86 MiB / 271.45 MiB 4.00% 1.05 Mi
3.96% 1.83 MiB > grc.io/k8s-minikube/kicbase... 5.16 MiB / 514.16 MiB 1.00% 527.39 K > preloaded-images-k8s-v18-v1... 11.14 MiB / 271.45 MiB 4.10% 1.07 Mi > grc.io/k8s-min
ikube/kicbase... 5.66 MiB / 514.16 MiB 1.10% 527.39 K > preloaded-images-k8s-v18-v1... 11.36 MiB / 271.45 MiB 4.18% 1.07 Mi > grc.io/k8s-min
ikube/kicbase... 6.16 MiB / 514.16 MiB 1.20% 556.72 K > preloaded-images-k8s-v18-v1... 11.70 MiB / 271.45 MiB 4.31% 1.07 Mi > grc.io/k8s-minikub
e/kicbase... 6.64 MiB / 514.16 MiB 1.29% 656.72 K > preloaded-images-k8s-v18-v1... 12.04 MiB / 271.45 MiB 4.44% 1.10 Mi > grc.io/k8s-minikube/kic
base... 7.36 MiB / 514.16 MiB 1.43% 656.72 K > preloaded-images-k8s-v18-v1... 12.30 MiB / 271.45 MiB 4.56% 1.10 Mi > grc.io/k8s-minikube/kicbas
e... 7.86 MiB / 514.16 MiB 1.53% 656.72 K > preloaded-images-k8s-v18-v1... 12.67 MiB / 271.45 MiB 4.69% 1.10 Mi > grc.io/k8s-minikube/kicbase...
9.46 MiB / 514.16 MiB 1.66% 813.71 K > preloaded-images-k8s-v18-v1... 12.89 MiB / 271.45 MiB 4.75% 1.12 Mi > grc.io/k8s-minikube/kicbase... 8
.85 MiB / 514.16 MiB 1.72% 813.71 K > preloaded-images-k8s-v18-v1... 13.08 MiB / 271.45 MiB 4.82% 1.12 Mi > grc.io/k8s-minikube/kicbase... 9.42
MiB / 514.16 MiB 1.83% 813.71 K > preloaded-images-k8s-v18-v1... 13.33 MiB / 271.45 MiB 4.91% 1.12 Mi > grc.io/k8s-minikube/kicbase... 9.69 MiB
14.16 MiB 1.98% 897.22 K > preloaded-images-k8s-v18-v1... 13.54 MiB / 271.45 MiB 4.99% 1.12 Mi > grc.io/k8s-minikube/kicbase... 10.18 MiB / 5
14.16 MiB 2.10% 897.22 K > preloaded-images-k8s-v18-v1... 13.70 MiB / 271.45 MiB 5.05% 1.12 Mi > grc.io/k8s-minikube/kicbase... 10.77 MiB / 514
.16 MiB 2.23% 897.22 K > preloaded-images-k8s-v18-v1... 13.96 MiB / 271.45 MiB 5.14% 1.12 Mi > grc.io/k8s-minikube/kicbase... 11.45 MiB / 514
.16 MiB 2.36% 897.22 K > preloaded-images-k8s-v18-v1... 14.21 MiB / 271.45 MiB 5.23% 1.12 Mi > grc.io/k8s-minikube/kicbase... 12.14 MiB / 514
.16 MiB 2.49% 897.22 K > preloaded-images-k8s-v18-v1... 14.46 MiB / 271.45 MiB 5.32% 1.12 Mi > grc.io/k8s-minikube/kicbase... 12.83 MiB / 514
.16 MiB 2.62% 897.22 K > preloaded-images-k8s-v18-v1... 14.71 MiB / 271.45 MiB 5.41% 1.12 Mi > grc.io/k8s-minikube/kicbase... 13.52 MiB / 514
.16 MiB 2.75% 897.22 K > preloaded-images-k8s-v18-v1... 14.96 MiB / 271.45 MiB 5.50% 1.12 Mi > grc.io/k8s-minikube/kicbase... 14.21 MiB / 514
.16 MiB 2.88% 897.22 K > preloaded-images-k8s-v18-v1... 15.21 MiB / 271.45 MiB 5.59% 1.12 Mi > grc.io/k8s-minikube/kicbase... 14.90 MiB / 514
.16 MiB 3.01% 897.22 K > preloaded-images-k8s-v18-v1... 15.46 MiB / 271.45 MiB 5.68% 1.12 Mi > grc.io/k8s-minikube/kicbase... 15.59 MiB / 514
.16 MiB 3.14% 897.22 K > preloaded-images-k8s-v18-v1... 15.71 MiB / 271.45 MiB 5.77% 1.12 Mi > grc.io/k8s-minikube/kicbase... 16.28 MiB / 514
.16 MiB 3.27% 897.22 K > preloaded-images-k8s-v18-v1... 15.96 MiB / 271.45 MiB 5.86% 1.12 Mi > grc.io/k8s-minikube/kicbase... 16.97 MiB / 514
.16 MiB 3.40% 897.22 K > preloaded-images-k8s-v18-v1... 16.21 MiB / 271.45 MiB 5.95% 1.12 Mi > grc.io/k8s-minikube/kicbase... 17.66 MiB / 514
.16 MiB 3.53% 897.22 K > preloaded-images-k8s-v18-v1... 16.46 MiB / 271.45 MiB 6.04% 1.12 Mi > grc.io/k8s-minikube/kicbase... 18.35 MiB / 514
.16 MiB 3.66% 897.22 K > preloaded-images-k8s-v18-v1... 16.71 MiB / 271.45 MiB 6.13% 1.12 Mi > grc.io/k8s-minikube/kicbase... 19.04 MiB / 514
.16 MiB 3.79% 897.22 K > preloaded-images-k8s-v18-v1... 16.96 MiB / 271.45 MiB 6.22% 1.12 Mi > grc.io/k8s-minikube/kicbase... 19.73 MiB / 514
.16 MiB 3.92% 897.22 K > preloaded-images-k8s-v18-v1... 17.21 MiB / 271.45 MiB 6.31% 1.12 Mi > grc.io/k8s-minikube/kicbase... 20.42 MiB / 514
.16 MiB 4.05% 897.22 K > preloaded-images-k8s-v18-v1... 17.46 MiB / 271.45 MiB 6.40% 1.12 Mi > grc.io/k8s-minikube/kicbase... 21.11 MiB / 514
.16 MiB 4.18% 897.22 K > preloaded-images-k8s-v18-v1... 17.71 MiB / 271.45 MiB 6.49% 1.12 Mi > grc.io/k8s-minikube/kicbase... 21.80 MiB / 514
.16 MiB 4.31% 897.22 K > preloaded-images-k8s-v18-v1... 17.96 MiB / 271.45 MiB 6.58% 1.12 Mi > grc.io/k8s-minikube/kicbase... 22.49 MiB / 514
.16 MiB 4.44% 897.22 K > preloaded-images-k8s-v18-v1... 18.21 MiB / 271.45 MiB 6.67% 1.12 Mi > grc.io/k8s-minikube/kicbase... 23.18 MiB / 514
.16 MiB 4.57% 897.22 K > preloaded-images-k8s-v18-v1... 18.46 MiB / 271.45 MiB 6.76% 1.12 Mi > grc.io/k8s-minikube/kicbase... 23.87 MiB / 514
.16 MiB 4.70% 897.22 K > preloaded-images-k8s-v18-v1... 18.71 MiB / 271.45 MiB 6.85% 1.12 Mi > grc.io/k8s-minikube/kicbase... 24.56 MiB / 514
.16 MiB 4.83% 897.22 K > preloaded-images-k8s-v18-v1... 18.96 MiB / 271.45 MiB 6.94% 1.12 Mi > grc.io/k8s-minikube/kicbase... 25.25 MiB / 514
.16 MiB 4.96% 897.22 K > preloaded-images-k8s-v18-v1... 19.21 MiB / 271.45 MiB 7.03% 1.12 Mi > grc.io/k8s-minikube/kicbase... 25.94 MiB / 5
```

Loading the Docker image into Minikube

I started the Kubernetes cluster by running the appropriate Minikube command and verified that the cluster was running successfully. Then, I loaded my Docker image into the Minikube environment so that Kubernetes could access it. After that, I ran `kubectl get nodes`, which showed the node status as Ready, confirming that the cluster was fully initialized and able to run containers.

Why load image into Minikube

Deploying to Kubernetes

In this step, I'm deployin my container using Kubernetes. I need a Deployment because that's how Kubernetes takes over application

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: rag-app-deployment
5 spec:
6   replicas: 1
7   selector:
8     matchLabels:
9       app: rag-api
10  template:
11    metadata:
12      labels:
13        app: rag-api
14    spec:
15      containers:
16        - name: rag-api-container
17          image: rag-app
18          imagePullPolicy: Never
19          ports:
20            - containerPort: 8000
21          env:
22            - name: OLLAMA_HOST
23              value: "host.docker.internal:11434"
```

How the Deployment keeps my app running

The deployment.yaml file tells Kubernetes how to run and manage your application as a Deployment. The key parts are the metadata, spec, selector, template, and container configuration. The image field specifies which container image Kubernetes should run for your app.



The replicas field means how many identical copies (pods) of your application should be running at the same time. After that opening idea, here's what your specific file is doing in simple terms: apiVersion: apps/v1 & kind: Deployment This tells Kubernetes you are creating a Deployment using the stable apps API. metadata.name: rag-app-deployment This is just the name of the Deployment inside the cluster. replicas: 1 Kubernetes will keep exactly one pod running. If it crashes, Kubernetes will recreate it. selector.matchLabels.app: rag-api This tells the Deployment which pods it owns. It must match the labels in the pod template.

```
abhinave@320DESKTOP-32UV: /mnt/c/Users/abhin/OneDrive/Desktop/IronFleet/Archives/Build a RAG API$ docker images
REPOSITORY          TAG                 IMAGE ID            SIZE
registry.k8s.io/coredns/coredns:v1.13.1                  aa5e3ebc8dfe       78.1MB
registry.k8s.io/etcd/v3.5.0                               8a186f718956       62.5MB
registry.k8s.io/kube-apiserver:v1.35.0                    5c6acd0749cd       80.8MB
registry.k8s.io/kube-controller-manager:v1.35.0          2c9a4b858bd7       75.8MB
registry.k8s.io/kube-proxy:v1.35.0                        32652f1bbe6        70.7MB
registry.k8s.io/kube-scheduler:v1.35.0                   55079ac3b12a       51.7MB
registry.k8s.io/pause:3.10.1                              cd073f4c5f6a       736kB

(abhinave@320DESKTOP-32UV) /mnt/c/Users/abhin/OneDrive/Desktop/IronFleet/Archives/Build a RAG API$ kubectl apply -f deployment.yaml
deployment.apps/rag-app-deployment created
(abhinave@320DESKTOP-32UV) /mnt/c/Users/abhin/OneDrive/Desktop/IronFleet/Archives/Build a RAG API$ kubectl get deployments
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
rag-app-deployment     0/1     1             0           54s
(abhinave@320DESKTOP-32UV) /mnt/c/Users/abhin/OneDrive/Desktop/IronFleet/Archives/Build a RAG API$ kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
rag-app-deployment-d8cd8d9d-vj4zh  0/1     ErrImageNeverPull  0           98s
(abhinave@320DESKTOP-32UV) /mnt/c/Users/abhin/OneDrive/Desktop/IronFleet/Archives/Build a RAG API$
```



Abhinave P.B

NextWork Student

nextwork.org

What did you observe when checking your pods?

The pod was created successfully, but Kubernetes could not start the container because it failed to find or pull the Docker image. As a result, the container is not running, and the pod is not ready.



Creating a Service

In this step, I'm creating a Kubernetes Service. I need a Service because it runs the app

What does the service.yaml file do?

The service.yaml file tells Kubernetes to create a stable network endpoint that exposes your RAG API and manages traffic to the correct Pods, even if those Pods restart or change IP addresses. The selector finds Pods by matching labels—specifically `app=rag-api`—so the Service knows exactly which Pods should receive incoming requests. This label must match the labels defined in your Deployment, otherwise no traffic will be routed. The port configuration allows the Service to listen on port 8000 and forward all incoming traffic to port 8000 inside the selected Pods, which is where the RAG API application is running. NodePort enables access from outside by opening a high-numbered port (between 30000–32767) on each node in the cluster and forwarding traffic from that port to the Service, making the API reachable from your local machine when using Minikube.



```
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ kubectl get nodes
E0205 20:29:54.116611 5329 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"https://127.0.0.1:65182/api?timeout=32s\": dial tcp 127.0.0.1:65182: connect: connection refused"
E0205 20:29:54.116947 5329 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"https://127.0.0.1:65182/api?timeout=32s\": dial tcp 127.0.0.1:65182: connect: connection refused"
E0205 20:29:54.118546 5329 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"https://127.0.0.1:65182/api?timeout=32s\": dial tcp 127.0.0.1:65182: connect: connection refused"
E0205 20:29:54.118797 5329 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"https://127.0.0.1:65182/api?timeout=32s\": dial tcp 127.0.0.1:65182: connect: connection refused"
E0205 20:29:54.120214 5329 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"https://127.0.0.1:65182/api?timeout=32s\": dial tcp 127.0.0.1:65182: connect: connection refused"
The connection to the server 127.0.0.1:65182 was refused - did you specify the right host or port?
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ kubectl apply -f service.yaml
error: error validating "service.yaml": error validating data: failed to download openapi: Get "https://127.0.0.1:65182/openapi/v2?timeout=32s": dial tcp 127.0.0.1:65182: connect: connection refused; if you choose to ignore these errors, turn validation off with --validate=false
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ kubectl config get-contexts
CURRENT  NAME  CLUSTER  AUTHINFO  NAMESPACE
*        minikube  docker-desktop  docker-desktop  minikube  default
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ minikube start
minikube v1.38.0 on Ubuntu 24.04 (kvm/amd64)
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.49 ...
* Restarting existing docker container for "minikube" ...
* Preparing Kubernetes v1.35.0 on Docker 29.2.0 ...
* Verifying Kubernetes components ...
  * Using image gcr.io/486-minikube/storage-provisioner:v5
  * Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ kubectl get nodes
minikube  Ready  control-plane  46h  v1.35.0
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ kubectl apply -f service.yaml
service/rag-app-service created
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ kubectl get services
NAME      TYPE          CLUSTER-IP  EXTERNAL-IP  PORT(S)  AGE
kubernetes  ClusterIP     10.96.0.1   <none>        443/TCP  46h
rag-app-service  NodePort     10.101.18.216  <none>        8000:31911/TCP  19s
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ kubectl get services
NAME      TYPE          CLUSTER-IP  EXTERNAL-IP  PORT(S)  AGE
kubernetes  ClusterIP     10.96.0.1   <none>        443/TCP  47h
rag-app-service  NodePort     10.101.18.216  <none>        8000:31911/TCP  48m
abhinave32@DESKTOP-32UV7P9: /mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$
```

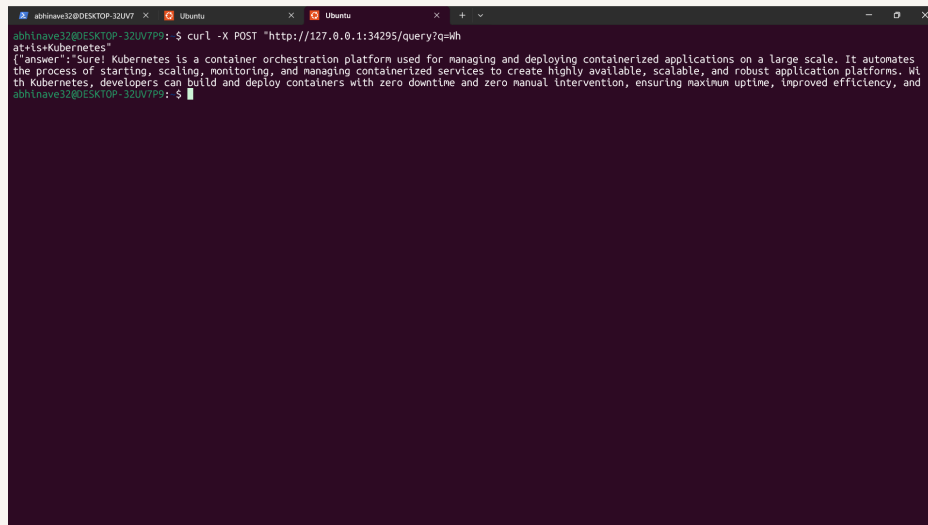
What kubectl commands did you run to create the service?

I applied my Service file by running `kubectl apply -f service.yaml`. I then verified that the Service was created by `kubectl get services`



Accessing My API Through Kubernetes

In this step, I'm testing my API is running in Kubernetes



```
abhinave32@DESKTOP-32U7P9: ~$ curl -X POST "http://127.0.0.1:34295/query?q=whatisKubernetes"
{"answer": "Sure! Kubernetes is a container orchestration platform used for managing and deploying containerized applications on a large scale. It automates the process of starting, scaling, monitoring, and managing containerized services to create highly available, scalable, and robust application platforms. With Kubernetes, developers can build and deploy containers with zero downtime and zero manual intervention, ensuring maximum uptime, improved efficiency, and"}
abhinave32@DESKTOP-32U7P9: ~$
```

How I accessed my API

I tested my API by running POST requests to the exposed service endpoint using curl. The response returned valid data from the /query and /add endpoints, indicating that the API was reachable and functioning as expected. This confirms that the application was successfully deployed in the Kubernetes cluster, the pod was running correctly, and the service was properly exposing the application through a NodePort. The main difference between Docker and Kubernetes deployment is that Docker focuses on running a single containerized application, while Kubernetes is responsible for orchestrating and managing multiple containers across a cluster.



Docker ensures that the application runs consistently inside a container, whereas Kubernetes handles scalability, service discovery, load balancing, self-healing, and rolling updates. In this deployment, Docker was used to package the RAG API, and Kubernetes ensured that the application remained available and accessible even in a distributed environment

Request flow through Kubernetes

The request flow went from my computer to the Kubernetes node using the node's IP address and NodePort. NodePort enabled external access by exposing the Service on a specific port of the node. The node then forwarded the request to the Kubernetes Service. The Service routed traffic by using its selector to identify the matching Pods and load-balancing the request to one of them. The selected Pod passed the request to the container where the API was running, and the response traveled back through the same path to the client.

Testing Self-Healing

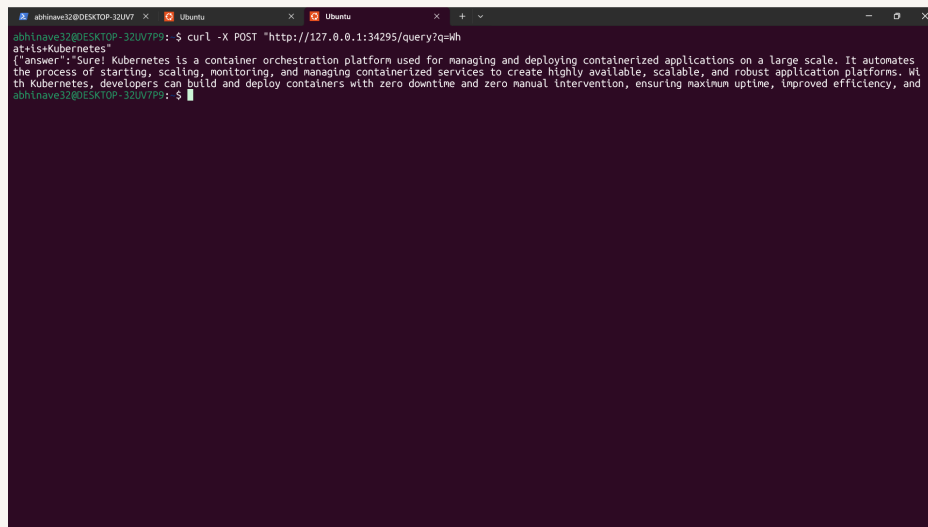
In this project extension, I'm demonstrating Kubernetes' self-healing capability by manually deleting a running pod and observing how the system automatically creates a replacement pod without any manual intervention. Even though the pod was terminated, Kubernetes continuously monitored the desired state defined in the Deployment and ensured that a new pod was started to maintain application availability. Self-healing is important because it allows applications to remain highly available and resilient to failures. In real-world systems, pods can fail due to crashes, resource limits, or node issues. Kubernetes detects these failures and automatically restarts or replaces the affected pods, ensuring minimal downtime. This reduces the need for manual monitoring and intervention, making applications more reliable and suitable for production environments.

```
abhinave32@DESKTOP-32UV7P9: $ kubectl get pods --watch
NAME                                READY   STATUS    RESTARTS   AGE
rag-app-deployment-5489bb7598-nk16r 1/1     Running   0          46m
rag-app-deployment-5489bb7598-nk16r 1/1     Terminating 0          47m
rag-app-deployment-5489bb7598-nk16r 1/1     Terminating 0          47m
rag-app-deployment-5489bb7598-lv4kw 0/1     Pending   0          0s
rag-app-deployment-5489bb7598-lv4kw 0/1     Pending   0          0s
rag-app-deployment-5489bb7598-lv4kw 0/1     ContainerCreating 0          0s
rag-app-deployment-5489bb7598-nk16r 0/1     Completed 0          47m
rag-app-deployment-5489bb7598-nk16r 1/1     Running   0          2s
rag-app-deployment-5489bb7598-nk16r 0/1     Completed 0          47m
rag-app-deployment-5489bb7598-nk16r 0/1     Completed 0          47m
```



What did you observe when you deleted the pod?

When I deleted the pod, I saw the status change from running to terminating to ContainerCreating. A new pod was created because of the Reconciliation loop.



```
abhinave32@DESKTOP-32UV7P9: ~$ curl -X POST "http://127.0.0.1:34295/query?q=whatisKubernetes"
{"answer": "Sure! Kubernetes is a container orchestration platform used for managing and deploying containerized applications on a large scale. It automates the process of starting, scaling, monitoring, and managing containerized services to create highly available, scalable, and robust application platforms. With Kubernetes, developers can build and deploy containers with zero downtime and zero manual intervention, ensuring maximum uptime, improved efficiency, and"}
abhinave32@DESKTOP-32UV7P9: ~$
```

How the Service routed traffic to the new pod

The Service automatically redirected traffic to the newly created pod because Kubernetes Services use label selectors to dynamically track healthy pods rather than relying on fixed IP addresses.



Abhinave P.B

NextWork Student

nextwork.org

When the original pod was deleted, the Service detected the replacement pod with the same labels and immediately began routing requests to it without requiring any configuration changes. Without Kubernetes, this would have required manual intervention, such as restarting the application, updating IP addresses, or reconfiguring networking and load-balancing rules, which could lead to downtime and service disruption. Self-healing is critical in production because applications must remain available despite unexpected failures. Kubernetes' ability to automatically detect failures and recover ensures high availability, fault tolerance, and consistent user experience, making it suitable for large-scale, real-world systems.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

